UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°1:

Introducción al SQL. Sentencias de Definición de Datos

Material Teórico

1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

¿QUÉ ES SQL? 2
Usos de SQL 2
Beneficios 4
¿QUÉ ES SQL SERVER? 5
Arquitectura cliente-servidor 5
Aplicaciones cliente 6
Modelos de almacenamiento de datos: 6
Objetos de SQL Server 7
TRANSACT-SQL (T-SQL) 8
Tipos de Sentencias 8
SENTENCIAS DE DEFINICIÓN DE DATOS – DDL 9
Creación de una tabla 10
Tipos de datos 11
Modificación de una tabla 17
Eliminación de una tabla 19
BIBLIOGRAFÍA 20

¿QUÉ ES SQL?

Imagen 1: Extraída de Banco de Imágenes Pixabay

SQL, **Lenguaje de Consulta Estructurado** (Structured Query Language) ***es un lenguaje declarativo de acceso a bases de datos relacionales que permite especificar diversas operaciones sobre las mismas***. Se trata de un lenguaje estándar utilizado para comunicarse con bases de datos relacionales, definido y publicado por el ISO (International Organization for Standardization) y el ANSI (American National Standards Institute).

Este lenguaje unifica características del álgebra y el cálculo relacional y permite lanzar consultas con el fin de recuperar información de interés de una base de datos. La sintaxis de SQL es sencilla y utiliza un lenguaje similar al inglés utilizado para la comunicación entre personas.

Es importante destacar que, aunque SQL es un lenguaje estándar, no es idéntico para cada base de datos. Cada motor de bases de datos tiene sus peculiaridades y, por lo tanto, puede haber algunas diferencias en el lenguaje SQL utilizado para interactuar con cada una de ellas.

A pesar de esto, cualquier sentencia escrita en ANSI SQL será interpretable por cualquier motor de bases de datos, lo que lo convierte en una herramienta muy útil y versátil para trabajar con datos en diferentes plataformas. En resumen, SQL es un lenguaje esencial para trabajar con bases de datos y se utiliza en una gran variedad de aplicaciones y sistemas.

Usos de SQL

SQL es utilizado en bases de datos relacionales que permite a los usuarios realizar una amplia variedad de operaciones en los datos almacenados en ellas. Algunos de los usos más comunes de SQL son:

- **Definición de datos**: SQL permite a los usuarios definir la estructura y organización de los datos almacenados en una base de datos, así como las relaciones existentes entre ellos. Al definir la estructura de la base de datos, los usuarios pueden especificar el tipo de datos que se almacenan en cada campo, su tamaño y otras propiedades, como si se requiere o no un valor único o si se permite que el campo sea nulo. Además, los usuarios también pueden crear tablas, vistas, índices y otros objetos de base de datos para ayudar en la gestión y organización de los datos.

- **Recuperación de datos**: SQL permite a los usuarios recuperar y utilizar los datos almacenados en una base de datos mediante la ejecución de consultas. Las consultas son declaraciones que especifican los datos que se desean recuperar y las condiciones que deben cumplir. SQL proporciona una amplia variedad de operadores para filtrar, ordenar, agrupar y resumir los datos, lo que permite a los usuarios obtener la información que necesitan de manera eficiente.
- **Manipulación de datos**: SQL también permite a los usuarios actualizar la base de datos mediante la adición de nuevos datos, la eliminación de los datos antiguos y la modificación de los datos existentes. Las declaraciones SQL que se utilizan para estas operaciones incluyen INSERT (para agregar nuevos registros), DELETE (para eliminar registros existentes) y UPDATE (para modificar los valores de los campos en los registros existentes).
- **Control de acceso**: SQL se utiliza para restringir la capacidad de los usuarios para acceder a los datos almacenados en una base de datos, protegiendo los datos contra accesos no autorizados. Los usuarios pueden definir permisos y roles de seguridad para limitar el acceso a la base de datos, lo que garantiza que solo los usuarios autorizados puedan acceder y manipular los datos.
- **Compartición de información**: SQL se utiliza para coordinar la compartición de datos entre usuarios concurrentes, asegurándose de que no haya interferencias entre ellos. SQL proporciona mecanismos para bloquear el acceso a registros y tablas mientras se están actualizando, lo que ayuda a prevenir conflictos y garantiza que los usuarios trabajen con datos actualizados.
- **Integridad de datos**: SQL permite a los usuarios definir restricciones de integridad en la base de datos, protegiéndola de alteraciones indebidas, actualizaciones inconsistentes o fallos de sistema. Las restricciones de integridad pueden ser de varios tipos, incluyendo restricciones de clave primaria y foránea, restricciones de unicidad y restricciones de comprobación. Al definir estas restricciones, los usuarios pueden garantizar que la base de datos se mantenga en un estado consistente y preciso en todo momento.

Beneficios

- **Independencia de los proveedores**: SQL es un lenguaje estándar utilizado por la mayoría de los sistemas de gestión de bases de datos (DBMS) relacionales, lo que significa que se puede cambiar de un DBMS a otro sin tener que reescribir completamente el código SQL. Además, esto reduce la dependencia de un proveedor específico y facilita la elección del DBMS que mejor se adapte a las necesidades de la organización.
- **Portabilidad entre sistemas informáticos**: Como SQL es un lenguaje independiente del sistema operativo y del hardware, las aplicaciones SQL se pueden transferir fácilmente de un sistema a otro, lo que permite a las organizaciones escalar sus sistemas de base de datos sin tener que rediseñar sus aplicaciones desde cero.
- **Fundamento Relacional**: SQL se basa en el modelo relacional, lo que significa que se pueden crear relaciones entre las tablas de una base de datos para reflejar mejor la estructura de los datos en la vida real. Esto hace que sea más fácil diseñar y mantener bases de datos complejas.
- **Estructura de alto nivel parecida al inglés**: Las sentencias SQL están diseñadas para que sean fáciles de leer y comprender, y utilizan una sintaxis que se asemeja al lenguaje natural en inglés. Esto hace que sea más fácil para los desarrolladores y los usuarios escribir y entender las sentencias SQL.
- **Consultas interactivas ad hoc** (en línea, al instante): SQL permite a los usuarios hacer consultas sobre los datos en tiempo real y obtener respuestas instantáneas. Esto es especialmente útil para los usuarios que necesitan realizar análisis ad hoc o informes personalizados sobre los datos.
- **Múltiples vistas de datos**: SQL permite a los usuarios crear vistas personalizadas de los datos, lo que significa que diferentes usuarios pueden ver la misma base de datos de diferentes maneras. Esto es útil cuando diferentes usuarios necesitan diferentes perspectivas de los datos.
- **Definición dinámica de datos**: SQL permite a los usuarios agregar, modificar y eliminar tablas y campos en tiempo real, incluso mientras los usuarios están accediendo a la base de datos. Esto permite a las organizaciones adaptarse rápidamente a los cambios en sus necesidades de datos.
- **Arquitectura cliente/servidor**: Las aplicaciones que utilizan SQL se comunican con los servidores de bases de datos a través de una red de área local. Esto permite a múltiples usuarios acceder a los mismos datos de manera simultánea y trabajar en colaboración en tiempo real.

¿QUÉ ES SQL SERVER?

SQL Server es un sistema de gestión de bases de datos relacionales (RDBMS, por sus siglas en inglés) desarrollado por Microsoft, que se utiliza para almacenar, administrar y recuperar grandes cantidades de datos. Es compatible con aplicaciones cliente/servidor, permitiendo que los usuarios accedan a los datos a través de una interfaz y una red LAN. SQL Server ofrece una amplia gama de características y herramientas para mejorar el rendimiento, la seguridad y la escalabilidad de la base de datos. Además, incluye una variedad de servicios, como la integración de datos, la generación de informes y la gestión de la seguridad, para simplificar la administración de la base de datos.

Arquitectura cliente-servidor

SQL Server utiliza la arquitectura cliente-servidor para separar la carga de trabajo en tareas que se ejecutan en equipos servidores y tareas que se ejecutan en equipos cliente. Esta arquitectura permite diseñar y distribuir aplicaciones para mejorar una gran variedad de entornos. Las interfaces de programación de los clientes proporcionan los medios para que las aplicaciones se ejecuten en equipos clientes independientes y se comuniquen con el servidor a través de la red.

La arquitectura cliente-servidor se compone de dos partes principales:

1. El **cliente**: es responsable de la lógica empresarial y de presentar los datos al usuario. Normalmente, el cliente se ejecuta en uno o varios equipos, pero también puede ejecutarse en el equipo servidor junto con SQL Server.
2. El **servidor**: administra las bases de datos y asigna los recursos disponibles del servidor (como la memoria, el ancho de banda de la red y las operaciones de disco) entre las distintas solicitudes. Además, el servidor es responsable de:
   - Mantener las relaciones entre los datos de una base de datos.
   - Asegurar que los datos estén correctamente almacenados y que no se infrinjan las reglas que definen las relaciones entre los datos.
   - Recuperar todos los datos hasta un punto de coherencia garantizada, en caso de que se produzca un error del sistema.

Para acceder al servidor desde el lado del cliente, se utiliza una aplicación basada en el estándar SQL, que utiliza un conjunto de sentencias y comandos que permiten especificar la información que se desea recuperar o modificar.

Aplicaciones cliente

Las aplicaciones cliente son el medio a través del cual los usuarios acceden a los datos almacenados en SQL Server. Estas aplicaciones proporcionan una interfaz que les permite interactuar con la base de datos sin necesidad de acceder directamente al servidor. SQL Server consta de dos componentes principales: el cliente y el servidor, cada uno con su función específica en el proceso de almacenamiento y recuperación de datos.

SQL Server utiliza una arquitectura cliente-servidor para separar la carga de trabajo en tareas que se ejecutan en equipos servidores y tareas que se ejecutan en equipos cliente. Además, utiliza una arquitectura de comunicación por capas para aislar las aplicaciones de la red y los protocolos subyacentes. Esto permite que las aplicaciones se distribuyan en diferentes entornos de red sin necesidad de cambiar su código fuente.

Modelos de almacenamiento de datos:

SQL Server administra bases de datos OLTP y OLAP:

- **Bases de datos OLTP** (On-Line Transactional Processing: procesamiento transaccional en línea) son bases de datos orientadas al procesamiento de transacciones que involucra acciones de inserción, eliminación y actualización de datos. SQL Server permite que un gran número de usuarios realice transacciones y modifique simultáneamente datos en tiempo real en bases de datos OLTP. Algunos ejemplos de bases de datos OLTP serían:
    - Sistemas de transacciones bancarias: donde los usuarios realizan depósitos, retiros y transferencias de dinero.
    - Sistemas de reservas de billetes de avión: donde los usuarios pueden reservar vuelos y comprar boletos.

- **Bases de datos OLAP** (On-Line Analytical Processing: procesamiento analítico en línea) involucran principalmente acciones de consulta. La tecnología OLAP organiza y resume grandes cantidades de datos, de manera que un analista pueda evaluar los datos rápidamente en tiempo real. Los datos son organizados de otra manera para permitir la realización de informes, análisis corporativos, estadísticas que ayudan a la toma de decisiones. Algunos ejemplos de bases de datos OLAP serían:

- Sistemas de reporte de ventas: donde se pueden obtener informes sobre las ventas de una empresa durante un período determinado.
- Sistemas de análisis de datos de redes sociales: donde se pueden obtener estadísticas sobre el uso de las redes sociales.

Objetos de SQL Server

En SQL Server, los datos se organizan en bases de datos que contienen una variedad de objetos para estructurar y mantener la integridad de los datos. Algunos de los objetos más comunes en SQL Server son:

Objeto	Definición	Ejemplo
Tabla	Es una estructura de datos que organiza los datos en filas y columnas y se utiliza para almacenar información de manera persistente en una base de datos.	Una tabla de empleados en la que cada fila representa un empleado y cada columna representa un atributo del empleado.
Vista	Es una tabla virtual que se basa en los resultados de una consulta. Se utiliza para simplificar la complejidad de una consulta o para restringir el acceso a los datos.	Una vista de empleados que solo muestra los nombres y salarios de los empleados sin mostrar sus direcciones o información personal.
Función	Es un objeto que acepta uno o más argumentos de entrada, realiza una tarea y devuelve un valor como resultado.	Una función que calcula la edad de un empleado a partir de su fecha de nacimiento.
Procedimiento almacenado	Es un grupo de instrucciones SQL que se almacenan como un objeto en la base de datos y se pueden llamar desde una aplicación o desde otra instrucción SQL.	Un procedimiento almacenado que inserta un nuevo empleado en la tabla de empleados.
Disparadores	Son objetos que se ejecutan automáticamente en respuesta a ciertos eventos, como la inserción, actualización o eliminación de datos en una tabla.	Un disparador que envía una notificación por correo electrónico cuando se inserta un nuevo empleado en la tabla de empleados.
Índice	Es una estructura de datos que mejora el rendimiento de las consultas de búsqueda en una tabla.	Un índice en la columna de apellido de la tabla de empleados para acelerar las consultas que buscan empleados por apellido.
Regla	Es una restricción personalizada que se aplica a una columna en una tabla.	Una regla que impide que se inserten empleados menores de edad en la tabla de empleados.
Restricción	Es una regla que se aplica a una o más columnas de una tabla y se utiliza para garantizar la integridad de los datos.	Una restricción que impide que se inserten empleados duplicados en la tabla de empleados.
Valor predeterminado	Es un valor que se asigna automáticamente a una columna si no se proporciona un valor durante la inserción de datos en una tabla.	Un valor predeterminado de 0 en la columna de salario de la tabla de empleados si no se proporciona un valor durante la inserción.

Tabla 1: Elaboración propia.

TRANSACT-SQL (T-SQL)

Transact-SQL (T-SQL) es un lenguaje incorporado en SQL Server que brinda una amplia gama de comandos y sentencias para administrar bases de datos. Con T-SQL, puedes crear, modificar y eliminar objetos como tablas, vistas, procedimientos almacenados y funciones. Además, puedes realizar operaciones de consulta, inserción, modificación y eliminación de datos en las tablas. T-SQL también ofrece funciones avanzadas, como el manejo de transacciones, el control de concurrencia y la programación de disparadores (triggers) y eventos.

Tipos de Sentencias

Tipos de Sentencias en SQL Server: Existen varios tipos de sentencias que se pueden ejecutar en SQL Server para interactuar con la base de datos. A continuación, se detallan algunos de los más importantes:

Tipos de Sentencias	Definición	Algunos ejemplos de las sentencias son:
Sentencias de Definición de Datos (DDL - Data Definition Language)	Son utilizadas para crear, modificar y eliminar objetos de la base de datos.	*CREATE TABLE*: se utiliza para crear una nueva tabla en la base de datos. *ALTER TABLE*: se utiliza para modificar la estructura de una tabla existente. *DROP TABLE*: se utiliza para eliminar una tabla de la base de datos. *CREATE INDEX*: se utiliza para crear un índice en una tabla.
Sentencias de Manipulación de Datos (DML - Data Manipulation Language)	Son utilizadas para recuperar, agregar, modificar o eliminar datos de una base de datos.	*SELECT*: se utiliza para recuperar datos de una o varias tablas. *INSERT*: se utiliza para agregar nuevos datos a una tabla. *UPDATE*: se utiliza para modificar datos existentes en una tabla. *DELETE*: se utiliza para eliminar datos de una tabla.
Sentencias de Control de Acceso (DCL - Data Control Language)	Son utilizadas para conceder o revocar privilegios a usuarios.	*GRANT*: se utiliza para conceder privilegios a un usuario. *REVOKE*: se utiliza para revocar los privilegios concedidos a un usuario.
Sentencias de Control de Transacciones (TCL - Transaction Control Language)	Son utilizadas para finalizar o abortar la transacción actual.	*COMMIT*: se utiliza para confirmar los cambios realizados en la base de datos en la transacción actual. *ROLLBACK*: se utiliza para deshacer los cambios realizados en la transacción actual y restaurar la base de datos a su estado anterior.
Sentencias de Programación	Son utilizadas para definir y manipular objetos de programación en la base de datos	*DECLARE*: se utiliza para definir una variable en una sentencia Transact-SQL. *OPEN*: se utiliza para abrir un cursor para procesar filas seleccionadas por una consulta. *EXECUTE*: se utiliza para ejecutar un procedimiento almacenado o una función definida por el usuario. *DESCRIBE*: se utiliza para obtener información sobre la estructura de una tabla o vista.

Tabla 2: Elaboración propia.

SENTENCIAS DE DEFINICIÓN DE DATOS – DDL

Las sentencias de definición de datos (DDL) en SQL son un conjunto de comandos que se utilizan para crear, modificar y eliminar objetos y propiedades de una base de datos, como tablas, vistas, procedimientos almacenados, restricciones, índices, reglas, entre otros.

Estas sentencias permiten al usuario definir la estructura y esquema de la base de datos, así como las relaciones entre las diferentes tablas y objetos. También pueden ser utilizadas para modificar y eliminar objetos existentes, como agregar o eliminar columnas de una tabla o cambiar el nombre de una vista.

Algunas de las sentencias DDL más comunes incluyen:

- ***CREATE*** se utiliza para crear un nuevo objeto en la base de datos.
- ***ALTER*** se utiliza para modificar un objeto existente
- ***DROP*** se utiliza para eliminar un objeto de la base de datos.

Es importante tener en cuenta que el uso adecuado de las sentencias DDL es esencial para garantizar la integridad y coherencia de la base de datos. Además, es importante tener en cuenta que algunas de estas sentencias pueden requerir permisos especiales para ser ejecutadas, lo que significa que no todos los usuarios pueden tener acceso a todas las funciones DDL de la base de datos.

Las sentencias principales son:

Sentencia	Objetos	Ejemplo
CREATE: Para crear un objeto	Tabla, Vista, Procedimiento almacenado, Función, Índice, Restricción, Regla, Disparador	**CREATE** *nombre_objeto*
ALTER: Para modificar un objeto	Tabla, Vista, Procedimiento almacenado, Función, Índice, Restricción, Regla, Disparador	**ALTER** *nombre_objeto*
DROP: Para eliminar un objeto	Tabla, Vista, Procedimiento almacenado, Función, Índice, Restricción, Regla, Disparador	**DROP** *nombre_objeto*

Tabla 3: Elaboración propia.

Creación de una tabla

La creación de una tabla es una operación fundamental en la gestión de bases de datos. Para crear una tabla en SQL, se utiliza la siguiente sintaxis de sentencia:

```
CREATE TABLE nombre_tabla
(nom_col1 tipo_dato [IDENTITY(inicio,incremento)][NULL|NOT NULL],
 nom_col2  tipo_dato,
 nom_col_n tipo_dato
 [CONSTRAINT nombre_constraint PRIMARY KEY(nom_col1)],
 [CONSTRAINT nombre_constraint FOREIGN KEY (nom_col2)
            REFERENCES tabla2 (nombre_columna_tabla2)]
)
```

En donde:

- **CREATE TABLE**: es la cláusula que indica que se va a crear una nueva tabla.
- **nombre_tabla**: es el nombre que se le dará a la tabla que se va a crear.
- **nom_col1, nom_col2, nom_col_n**: se definen los nombres de las columnas de la tabla y su tipo de dato. Si hay más de una columna, se separan con comas.
- **IDENTITY(inicio, incremento)**: es una propiedad opcional que se utiliza para definir una columna como una columna identidad. Esta columna genera automáticamente un valor único e incremental, comenzando en el valor especificado en *"inicio"* y aumentando en *"incremento"*. Solo puede asignarse a tipos de datos numéricos y puede haber solo una columna identidad por tabla. Se utiliza comúnmente en conjunto con la clave primaria (PRIMARY KEY).
- **NULL|NOT NULL**: se utiliza para indicar si una columna permite o no valores nulos.
- **CONSTRAINT**: es una cláusula opcional que se puede utilizar para agregar una restricción a la tabla, como una restricción de clave primaria o una restricción de clave foránea.
    - **PRIMARY KEY**: es una restricción que se utiliza para identificar de manera única cada fila en la tabla. Solo se puede tener una clave primaria por tabla.

- **FOREIGN KEY**: es una restricción que se utiliza para establecer una relación entre dos tablas. La clave foránea hace referencia a una columna en otra tabla que es la clave primaria de esa tabla.
- **REFERENCES**: es una cláusula que se utiliza en combinación con la restricción FOREIGN KEY para establecer la relación entre dos tablas.

- Lo que está entre corchetes **[ ]** es opcional y puede incluirse o no en la sentencia.
- Lo que se separa por la barra vertical "**|**" indica que se debe elegir uno u otro.
- Las palabras en mayúsculas son palabras reservadas del lenguaje SQL y deben escribirse tal como se presenta en la sintaxis. No se deben utilizar para nombres de otros elementos como objetos o campos.
- El Management Studio de SQL Server **no es "case sensitive"**, lo que significa que no distingue entre mayúsculas y minúsculas, por lo que la sentencia puede escribirse completamente en minúsculas.

Tipos de datos

La creación de una base de datos implica la definición de sus tablas (también conocidas como entidades) y los campos que las componen. Sin embargo, antes de introducir cualquier dato en estos campos, es importante considerar la naturaleza de la información que se va a almacenar. Esto se debe a que una base de datos puede contener una amplia variedad de tipos de datos y, por lo tanto, es esencial especificar qué tipo de valor se va a introducir en cada campo.

Especificar el tipo de datos de un campo permite que la base de datos facilite la búsqueda de información posteriormente y optimice los recursos de memoria. Por ejemplo, si se desea almacenar precios de productos en una columna, se debería especificar que el tipo de datos sea numérico, específicamente moneda. Por otro lado, si se desea almacenar nombres, se debe escoger un tipo de dato que permita almacenar información de tipo carácter.

En resumen, la especificación del tipo de datos para cada campo es crucial para garantizar la integridad de la base de datos y asegurar una gestión de la información adecuada y eficiente.

Cada tipo de dato en SQL Server tendrá un nombre propio, algunos de los cuales se muestran en la tabla siguiente:

Tipo genérico	Tipo de dato almacenado	Nombre en SQL Server	Cantidad de bytes que ocupa en el almacenamiento
Binario	Almacenan cadenas de bits expresados en números hexadecimales.	**binary**	Longitud fija (hasta 8 Kb)
		varbinary	Puede variar en el nro. de dígitos hexadecimales (hasta 8 Kb.)
		image	Longitud variable y puede exceder los 8 Kb.
Alfanumérico o carácter	Combinación de letras, símbolos y números.	**char**	Longitud fija (hasta 8 Kb)
		varchar	Puede variar el nro. de caracteres (hasta 8Kb.)
		text	Pueden ser caracteres ASCII y exceder los 8 Kb.
Fecha y Hora	Fechas y horas (día, mes y año y hora, minuto y segundos según el formato del sistema	**datetime**	Fechas entre el 01/01/1753 hasta 31/12/9999 (8 by)
		smalldatetime	Fechas entre el 01/01/1900 hasta 06/06/2079 (4 by)
Numéricos Decimal (punto fijo)	Números con decimales de coma fija.	**decimal**	Pueden tener hasta 38 dígitos
		numeric	Equivalente al decimal.
Numéricos Decimal (punto flotante)	Números con decimales aproximados	**float**	Entre 1,79E + 308 hasta 1,79 + 308
		real	Entre 3,40E + 38 hasta 3,40 + 38.
Numéricos Enteros	Datos numéricos enteros positivos o negativos. Tinyint no tiene negativos	**bigint**	Entre 2^63 hasta 2^63 - 1 (8 by)
		int	Entre -2,147,483,648 hasta 2,147,483,648 (4by)
		smallint	Entre -32,768 hasta 32,767 (2by)
		tinyint	Entre 0 y 255 (1 by)
Numérico Moneda	Numérico decimal para cantidades monetarias positivas o negativas	**money**	Entre 922,337,203,685,477,5808 positivos y negativos (8 by)
		smallmoney	Entre 214,748,3648 (4 by)
	Especiales	**bit**	Datos que consisten de 1 o 0. Similar al True o False
		cursor	Es empleado en variables o procedimientos almacenados.
		timestamp	Indica la actividad que ocurre sobre una fila.
		uniqueidentifier	Es un identificador único de la fila dentro de la base de datos
		sql_variant	Almacena varios tipos de datos excepto el tect, ntext, timestamp, image.
		table	Almacena un resultado de una consulta.
Unicode	Se puede almacenar datos Unicode. Utilizan 2 byte por cada carácter	**nchar**	Longitud fija hasta 4000 caracteres Unicode.
		nvarchar	Longitud variable hasta 4000 caracteres unicode
		Ntext	Exceden los 4000 caracteres unicodes.

Tabla 4: Elaboración propia.

Ejemplo de creación de tablas

En este ejemplo se desea crear dos tablas, PERSONAS y TPOS_DNI, cada una con su respectiva estructura.

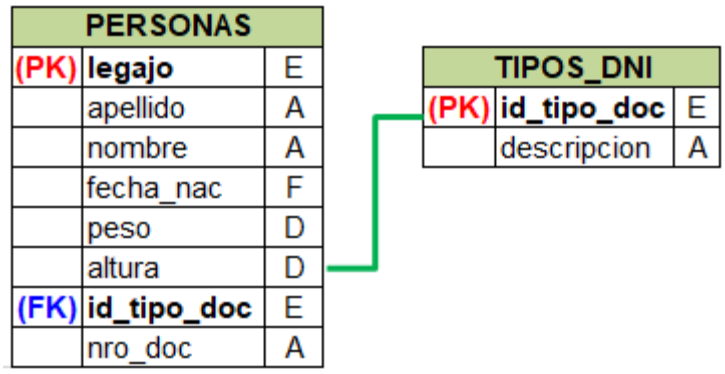

Gráfico 1: Elaboración propia.

Para asegurar la integridad referencial, es necesario crear primero la tabla TPOS_DNI, ya que su clave primaria será referenciada por una clave foránea en la tabla PERSONAS:

```
CREATE TABLE tipos_dni
(id_tipo_doc int IDENTITY (1,1),
 descripcion varchar(15)
 CONSTRAINT pk_tipo_doc PRIMARY KEY (id_tipo_doc)
)

CREATE TABLE personas
(legajo int,
 apellido varchar(50) not null,
 nombre varchar(50),
 fecha_nac datetime,
 peso decimal(4,2),
 altura decimal(3,2),
 id_tipo_doc int,
 nro_doc varchar(8)
 CONSTRAINT pk_persona PRIMARY KEY (legajo),
 CONSTRAINT fk_persona_tipo_doc  FOREIGN KEY (id_tipo_doc)
            REFERENCES tipos_dni (id_tipo_doc)
)
```

Explicación de la Tabla **TIPOS_DNI:**

TIPOS_DNI	
id_tipo_doc	descripcion
1	DNI
2	Pasaporte
3	LC
4	...

La estructura que compartiste corresponde a la creación de una tabla llamada "tipos_dni" que se utiliza para almacenar información sobre los diferentes tipos de documentos de identidad (por ejemplo, DNI, pasaporte, LC, carnet de identidad, etc.).

Aquí te explico los elementos y campos que conforman la estructura de la tabla:

- **CREATE TABLE**: indica que se va a crear una nueva tabla en la base de datos.
- **tipos_dni**: es el nombre de la tabla que se va a crear.
- **id_tipo_doc**: es el nombre del primer campo de la tabla. Es de tipo *"int"*, lo que significa que es un número entero. Además, se utiliza la **propiedad IDENTITY (1,1)**, lo que significa que el valor de este campo se incrementa automáticamente de uno en uno a medida que se agregan nuevos registros a la tabla. *Esto garantiza que cada registro tenga un valor único y consecutivo en este campo*.
- **descripcion**: es el nombre del segundo campo de la tabla. Es de tipo *"varchar(15)"*, lo que significa que puede almacenar una cadena de caracteres de hasta 15 caracteres de longitud. Este campo se utiliza para almacenar la descripción del tipo de documento de identidad.
- La definición de la clave primaria:

  ```
  CONSTRAINT pk_tipo_doc PRIMARY KEY (id_tipo_doc)
  ```

  Este es un `CONSTRAINT` (restricción) que se utiliza para definir la clave primaria (PK) de la tabla. La clave primaria es un campo (o conjunto de campos) que se utiliza para identificar de manera única cada registro en la tabla. En este caso, la clave primaria está definida por el campo "id_tipo_doc".

  El **"pk_tipo_doc"** es el nombre de la restricción, ***que puede ser cualquiera que no exista previamente como objeto en la misma base de datos*** y se utiliza para garantizar que no se puedan ingresar registros duplicados en la tabla en función de la clave primaria.

  A las claves primarias no es necesario indicarle **"not null"** ya que una vez definida la constraint primary key por definición no aceptará nulos.

En resumen, la tabla "tipos_dni" tiene dos campos: "id_tipo_doc", que es un número entero único que se genera automáticamente, y "descripcion", que es una cadena de caracteres que almacena la descripción del tipo de documento de identidad. Además, se define la clave primaria de la tabla como el campo "id_tipo_doc" para garantizar la unicidad de cada registro.

Explicación de la Tabla **PERSONAS**:

Esta estructura es una creación de tabla llamada "personas". A continuación, se explica cada campo y propiedad:

- **legajo**: es un campo de tipo entero que servirá como identificador único para cada persona registrada en la tabla.
- **apellido**: es un campo de tipo texto con una longitud máxima de 50 caracteres que almacena el apellido de la persona. La cláusula "**not null**" indica que este campo no puede estar vacío.
- **nombre**: es un campo de tipo texto con una longitud máxima de 50 caracteres que almacena el nombre de la persona.
- **fecha_nac**: es un campo de tipo fecha y hora que almacena la fecha de nacimiento de la persona.
- **peso**: es un campo de tipo decimal que almacena el peso de la persona con una precisión de 4 dígitos y 2 decimales. Por ejemplo, puede almacenar un valor de 58.25.
- **altura**: es un campo de tipo decimal que almacena la altura de la persona con una precisión de 3 dígitos y 2 decimales. Por ejemplo, puede almacenar un valor de 1.75.
- **id_tipo_doc**: es un campo de tipo entero que almacena el identificador del tipo de documento de la persona. Este campo es una clave foránea que se utiliza para establecer una relación con la tabla *"tipos_dni"*.
- **nro_doc**: es un campo de tipo texto con una longitud máxima de 8 caracteres que almacena el número de documento de la persona.

Además, la tabla "personas" tiene **dos restricciones**:

- La definición de la clave primaria:

```
CONSTRAINT pk_persona PRIMARY KEY (legajo),
```

  - **pk_persona**: el nombre de la `CONSTRAINT` ***que puede ser cualquiera que no exista previamente como objeto en la misma base de datos***, pero, como los nombres de las tablas, debe hacer referencia a lo que es. Luego va el tipo de restricción `PRIMARY KEY` y por último el campo sobre el cual se crea la clave primaria entre paréntesis.
  - Es una restricción de clave primaria que indica que el campo "legajo" es una clave única e identifica de manera exclusiva cada registro en la tabla.
  - Es importante destacar que la clave primaria pk_persona n**o tiene la propiedad IDENTITY**, lo que significa que los valores de la columna legajo no se generan automáticamente. Por lo tanto, al insertar datos en la tabla personas, se debe proporcionar explícitamente un valor para la columna legajo.
  - Las comas se utilizan para separar elementos que son del mismo tipo por ello va separando los campos y luego de la constraint primary key.

- En la definición de la clave foránea:

```
CONSTRAINT fk_persona_tipo_doc FOREIGN KEY (id_tipo_doc)
           REFERENCES tipos_dni (id_tipo_doc)
```

  - **fk_persona_tipo_doc**: el nombre de la `CONSTRAINT` ***que puede ser cualquiera que no exista previamente como objeto en la misma base de datos***. Luego va el tipo de restricción `FOREIGN KEY` , seguido por el campo sobre el cual se crea la clave foránea entre paréntesis, posterior a ello, (puede ser sobre la misma línea, aquí no entro en el ancho de la hoja) `REFERENCES` la tabla que contiene la clave primaria que referencia y por último el campo, entre paréntesis, que es PK de la tabla que es extremo uno en la relación.
  - Es una restricción de clave foránea que establece una relación entre el campo "id_tipo_doc" de la tabla "personas" y el campo "id_tipo_doc" de la tabla "tipos_dni". Esta restricción garantiza la integridad referencial, lo que significa que solo se permitirán valores en el campo "id_tipo_doc" que correspondan a un valor existente en la tabla "tipos_dni".

Modificación de una tabla

La modificación de una tabla en una base de datos es un proceso común que se lleva a cabo cuando se requiere realizar cambios en su estructura o definición. La sentencia **ALTER** permite realizar estas modificaciones y puede ser utilizada para agregar, eliminar o modificar componentes de una tabla, como restricciones, columnas, nombres de columnas, tipos de datos, etc.

Al modificar una tabla, se pueden hacer cambios en su estructura para adaptarla a nuevas necesidades o para corregir errores. Por ejemplo, se puede agregar una nueva columna para almacenar información adicional, cambiar el tipo de datos de una columna existente, agregar una restricción para garantizar la integridad de los datos, etc.

Además, la sentencia **ALTER** también puede utilizarse para eliminar restricciones o columnas que ya no son necesarias, cambiar el nombre de una columna para hacerlo más descriptivo o cambiar el nombre de la tabla en sí.

Es importante destacar que, al realizar modificaciones en una tabla, se deben tener en cuenta las consecuencias que puedan tener en otras partes del sistema que dependan de ella.

Para agregar una columna:

Para agregar una columna a una tabla se utiliza la sentencia **ALTER TABLE**, seguida del nombre de la tabla y la cláusula **ADD**, seguida del nombre y tipo de dato de la columna a agregar.

```
ALTER TABLE nombre_tabla
ADD nombre_columna  tipo_dato
```

Por ejemplo, si se quiere agregar la columna "contextura_fisica" a la tabla "personas" con un tipo de dato "varchar(20)", se utilizaría la siguiente sentencia:

```
ALTER TABLE personas
ADD contextura_fisica varchar(20)
```

Para eliminar una columna:

Para eliminar una columna de una tabla se utiliza la sentencia **ALTER TABLE**, seguida del nombre de la tabla y la cláusula **DROP COLUMN**, seguida del nombre de la columna a eliminar.

```
ALTER TABLE  nombre_tabla
DROP COLUMN  nombre_columna
```

Por ejemplo, si se quiere eliminar la columna "contextura_fisica" de la tabla "personas", se utilizaría la siguiente sentencia:

```
ALTER TABLE personas
DROP COLUMN contextura_fisica
```

Para modificar el tipo de dato:

Para modificar el tipo de dato de una columna en una tabla se utiliza la sentencia **ALTER TABLE**, seguida del nombre de la tabla y la cláusula **ALTER COLUMN**, **seguida del nombre y tipo de dato de la columna a modificar**.

```
ALTER TABLE nombre_tabla
ALTER COLUMN nombre_columna tipo_dato
```

Por ejemplo, si se quiere modificar el tamaño de la columna "nro_doc" de la tabla "personas" a un tipo de dato "varchar(10)", se utilizaría la siguiente sentencia:

```
ALTER TABLE personas
ALTER COLUMN nro_doc varchar(10)
```

Para agregar una clave primaria:

Para agregar una clave primaria a una tabla se utiliza la sentencia ALTER TABLE, seguida del nombre de la tabla y la cláusula ADD CONSTRAINT, seguida del nombre de la clave primaria y la cláusula PRIMARY KEY, seguida del nombre de la columna que será la clave primaria.

```
ALTER TABLE nombre_tabla
ADD CONSTRAINT nombre_clave PRIMARY KEY (columna_PK)
```

Por ejemplo, si se quiere agregar una clave primaria a la tabla "títulos" con la columna "id_titulo", se utilizaría la siguiente sentencia:

```
ALTER TABLE titulos
ADD CONSTRAINT pk_titulos PRIMARY KEY (id_titulo)
```

Para eliminar una constraint (clave primaria en este caso):

```
ALTER TABLE nombre_tabla
DROP CONSTRAINT nombre_clave;
```

Por ejemplo, si se quiere eliminar la constraint pk_titulos de la tabla "títulos", se utilizaría la siguiente sentencia:

```
ALTER TABLE titulos
DROP CONSTRAINT pk_titulos; //El nombre de la contraint
```

Eliminación de una tabla

La eliminación de una tabla en una base de datos se realiza utilizando la sentencia SQL **"DROP TABLE"**. Esta sentencia permite borrar definitivamente la tabla especificada, así como todo su contenido.

La sintaxis básica de la sentencia **DROP TABLE** es la siguiente:

```
DROP TABLE nombre_table
```

Por ejemplo, si deseas eliminar una tabla llamada "personas", simplemente deberías escribir:

```
DROP TABLE personas
```

Es importante tener en cuenta que la eliminación de una tabla también borra todos los datos que contiene, por lo que se debe tener precaución al utilizar esta sentencia, ya que los datos no pueden ser recuperados después de la eliminación.

Algunos puntos adicionales, como:

- Asegúrate de hacer una copia de seguridad de los datos importantes antes de eliminar cualquier tabla.
- Es posible que se requieran permisos especiales para eliminar una tabla, dependiendo del sistema de gestión de base de datos que estés utilizando.
- Al eliminar una tabla, también se eliminan todas las restricciones, índices y dependencias que estén relacionadas con ella. Por lo tanto, debes asegurarte de revisar cualquier otra tabla o objeto que pueda verse afectado por la eliminación

BIBLIOGRAFÍA

- Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK
- Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016
- Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill
- Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press
- Imágenes extraídas del Banco de Datos Gratuito Pixabay. Disponible en: https://pixabay.com/es/

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.